

استغرق التفكير 2m 46s

تقرير CTO الشامل

Inventory / Vouchers / Van Sales — مسار إنقاذ RAWAEA ERP

من بداية المهمة حتى نقطة التوقف

حالة التقرير: مراجعة رجعية مستقلة

حالة التنفيذ: متوقف بناءً على طلبك

المستودع: papamohammed77-glitch/raawaie-erp-review

الفرع الأخير المستخدم: rescue/manual-vouchers-inventory-core

ملاحظة منهجية مهمة: لم أعتبر أن كل ما في المستودع «حقيقة Production». فرّقت بين وثائق المشروع، كود الـ RPC، Evidence، مأخوذ من Production، وتقارير المساعدين. كما أن بعض المسارات التي ذكرتها الوثائق القديمة لم تعد موجودة بالاسم نفسه في main، وهو يحد ذاته جزء من مشكلة الـ contract drift.

1. الهدف الأصلي للمهمة

كانت المهمة أكبر من إصلاح خطأ في [vouchers.html](#).

الهدف كان الوصول إلى **Inventory Core مستقر وقابل للاعتماد** ثم إكمال:

- [vouchers.html](#)
- Manual Stock Voucher lifecycle
- Inventory movement engine

- Partial Receive
- Transfer
- Direct Sale / Direct Return
- Loading / Unloading
- ثم van-sales.html
- وربط Edge Functions ذات العلاقة
- مع الحفاظ على سلامة:
- o stock_branches
- o inventory_log
- o allocated_qty
- o Voucher lifecycle
- o Audit
- o Accounting/ledger paths
- o RLS/security

وثيقة البداية الرسمية للمشروع تصف النظام كـ Cloud-Native/Offline-First ERP، وتذكر 26 تطبيق PWA و 71 Edge Functions وعدداً كبيراً من جداول PostgreSQL، مع اعتبار Inventory/Warehouse جزءاً مركزياً من النظام.

2. المرحلة التي سبقت مهمة الإنقاذ

قبل الدخول في مشكلة Manual Vouchers كان التشخيص المعماري الأهم قد ظهر:

Distributed Business Logic

كانت العمليات التجارية موزعة على عدة Edge Functions، بحيث أكثر من مكان يستطيع:

- تعديل المخزون.
- إنشاء inventory_log.
- تعديل العهدة.

- إنشاء قيود مالية.
- تحديث Ledgers.

وهذا خلق **Multiple Sources of Truth**.

أخطر مثال تم اكتشافه:

Van Sales

كان هناك خطر أن تتم:

MAIN → VAN

ثم يحدث خصم آخر من MAIN أو من VAN بطريقة غير متسقة.

أي أن المشكلة لم تكن UI فقط؛ كانت في مكان امتلاك Business Logic نفسه.

3. الخطة المعمارية الأصلية للإصلاح

تم اقتراح بناء طبقة موحدة:

`post_stock_movement`

لتكون الجهة الوحيدة التي تنفذ:

Stock validation

↓

Row locking

↓

stock_branches update

↓

inventory_log

↓

audit

والـ Edge Functions لا تعدل المخزون مباشرة، بل تستدعي هذا المحرك.

وكانت العمليات المستهدفة تشمل:

PurchaseIn
TransferOut
TransferIn
Loading
Unloading
POSSale

VanSale
SalesReturn
PurchaseReturn
InventoryIncrease
InventoryDecrease
Adjustment

مع فصل:

allocated_qty

عن حركة المخزون لأنه **Reservation** وليس **Movement**.
ثم اقترحنا بنفس المنطق:

post_journal_entry
post_ledger_entry

لتجنب تكرار المشكلة ماليًا.

4. لماذا انتقلنا إلى Manual Vouchers؟

لأن vouchers.html أصبح نقطة حرجة لا يمكن إكمال بقية Inventory فوقها دون معرفة:

- كيف ينشأ Voucher.
- كيف يُرسل.
- كيف يُستلم.
- كيف يُكمل.
- كيف يُلغى.
- كيف تتحرك البضاعة.
- كيف تمنع إعادة نفس العملية.
- من يملك البضاعة في كل مرحلة.

وبالتالي تم إيقاف التوسع في vouchers.html و van-sales.html إلى حين تثبيت Inventory Core.

5. أول مشكلة كبيرة: Company Context

ظهر اختبار:

MANUAL_VOUCHER_LIFECYCLE

وفشل برسالة:

سياق الشركة غير متسق مع إعدادات النظام

فبدأ التحقيق في:

- app_settings
- branches
- company_id
- main_branch_id
- RLS

وكان من الطبيعي وقتها الاشتباه في RLS.

لكن Evidence أثبت لاحقًا:

```
app_settings.company_id
=
main_branch.company_id
```

وأن:

main_branch_id → BR-01

متسق مع الشركة.

كما أن app_settings كانت تحتوي Policy واسعة:

```
Allow all for all
qual = true
```

لذلك لم يكن RLS هو السبب الأساسي المثبت.

هذه المرحلة استهلكت وقتًا لأننا لم نمتلك منذ البداية Production Contract موحدًا يجمع هذه الحقائق.

6. المشكلة الحقيقية الأولى: completed_by

بعد تجاوز مشكلة Context ظهر الخطأ الحقيقي:

```
column "completed_by" of relation "stock_vouchers" does not exist
```

والـ Production Schema أثبت أن stock_vouchers يحتوي:

```
id
company_id
voucher_code
voucher_date
type
status
from_branch_id
to_branch_id
from_type
from_id
to_type
to_id
reference
notes
created_by
sent_date
received_date
completed_at
created_at
updated_at
source
```

ولا يحتوي:

```
completed_by
```

بينما complete_manual_stock_voucher_atomic () كان ينفذ:

SQL

```
UPDATE stock_vouchers
SET
  status = 'Completed',
  completed_at = now(),
  completed_by = p_user_email
...
```

هذا كان تعارضًا حقيقيًا بين RPC و Production Schema.

7. المشكلة الأعمق خلف completed_by

المشكلة ليست أن column واحدًا ناقص.

المشكلة هي:

RPC Contract

=/

Production Schema Contract

وهذا يعني أن عملية التطوير كانت تسمح بوجود:

Code assumes future schema

بينما Production يحتوي:

Actual schema

دون Gate يمنع النشر.

ولهذا فإن إضافة:

SQL

completed_by

كانت ستصلح الخطأ المباشر، لكنها لن تعالج Root Cause.

8. التحقيق في inventory_log

ظهرت مشكلة ثانية:

الوثائق/التصور المعماري كانت تتعامل مع:

inventory_log.branch_id

بينما Production لا يحتوي هذا العمود.

وهنا كان القرار الصحيح الذي وصلنا إليه:

لا نضيف branch_id فقط لأن وثيقة قديمة تشير إليه.

يجب أولاً إثبات أن الـ Business Model يحتاجه.

وهذه نقطة مهمة لأن إضافة column غير ضروري إلى Core Inventory قد تزيد التعقيد بدل إصلاحه.

9. اكتشاف Partial RECEIVE

وهنا ظهرت المشكلة الأخطر من completed_by.

ال Voucher يسمح بالاستلام الجزئي.

مثل:

Ordered = 100

Receive 30

Receive 20

Receive 50

وهذا صحيح.

لكن عند Retry بسبب:

- timeout
- network failure
- client retry
- duplicate request

يمكن أن يصبح:

Receive 30

Receive same request 30

فتصبح:

received_qty = 60

رغم أن العملية الثانية لم تكن Business Operation جديدة.

10. لماذا FOR UPDATE وحده لا يكفي؟

كنا بحاجة إلى التفريق بين شيئين:

Concurrency

منع:

A → 60

B → 60

من تجاوز:

100

وهذا يحتاج:

SQL

FORUPDATE

Idempotency

منع:

Request A

Request A again

من تسجيل عمليتين.

وهذا لا يحله FOR UPDATE وحده.

لذلك أصبح لدينا مساران مستقلان:

Concurrency Control

+

Idempotency Control

11. Evidence الخاصة بالIndexes

تم التحقق من stock_voucher_details.

والEvidence الفعلية أظهرت أن الفهرس الموجود كان أساسًا:

PRIMARY KEY(id)

ولا يوجد Unique mechanism يمنع Duplicate Receive Operations.

وهذا أكد أن:

Idempotency يجب أن تكون جزءًا من تصميم العملية نفسها، وليس شيئًا نتوقع أن يمنع Schema الحالي تلقائيًا.

12. مشكلة Custody

ظهرت مسألة أكثر عمقًا:

DirectSale

هل هو:

Branch → Customer

أم:

Branch → VAN → Customer

؟

DirectReturn

هل هو:

Customer → Branch

أم:

Customer → VAN → Branch

؟

هذه ليست مشكلة SQL.

هذه **Business Custody Decision**.

ولو تم اختيارها خطأ فسنحصل على:

double deduction

أو:

phantom VAN stock

أو:

wrong branch stock

وهذا بالذات سبب أن بعض اقتراحات المستشارين كانت صحيحة تقنيًا لكنها غير صالحة للاعتماد مباشرة.

13. مشكلة app_settings و RLS

تم التحقيق فيها بشكل مستقل لأن الرسالة الأولى كانت:

سياق الشركة غير متسق مع إعدادات النظام

لكن النتيجة النهائية:

- RLS مفعل.
- توجد Policy واسعة.
- app_settings متسقة مع Branch.
- main_branch_id صحيح.

وبالتالي:

لم يتم إثبات أن RLS هو Root Cause.

إعادة فتح هذه النقطة مرات إضافية كانت من الأشياء التي ساهمت في الدوران.

14. مرحلة Evidence Collection

تم إنشاء مجموعة كبيرة من ملفات Evidence، منها:

SQL_Evidence/diagnostics/

وتضمنت:

- 1) Production schema.csv
 - 2) All manual-voucher RPC definitions.csv
 - 3) RPC privileges.csv
 - 4) Detect documentedRPC column drift.md
 - 5) Inventory-log actual contract.csv
 - 6) Branchcompany consistency.csv
 - 7) Settingscompanymain-branch consistency.csv
 - 8) Stock availability by branch.csv
- التعريفات.csv

الفهارس الفعلية
المد حول التعريفات

هذه الملفات كانت مفيدة جدًا في النهاية لأنها حولت جزءًا من المشكلة من تخمين إلى Evidence.

15. ما أثبتته Evidence فعليًا

بحسب الملفات التي راجعناها:

Production

Company:
da4ef704-88ac-4120-aa0e-65b92b2aa2bc

Branches

الفرع الرئيسي — BR-01
الاسكندرية — BR-2

وكلاهما تحت نفس الشركة.

Main Branch

BR-01
151e5cd7-ac4a-4fc3-b703-d73a0dbb0dc6

Inventory

BR-01 → 8624 available
BR-2 → 0

stock_vouchers
لا يوجد:

completed_by

inventory_log
لا يوجد:

branch_id

RPC

complete_manual_stock_voucher_atomic يعتمد على:

completed_by

Partial Receive

لا يوجد Unique operation identity كافٍ لمنع Replay.

16. دخول المساعدين

بعد أن أصبحت المشكلة أكبر من إصلاح واحد، تم إنشاء نموذج Team:

Hussein

دوره:

Primary Implementer / Architecture Executor

يقرأ التكليف ثم:

- يحلل.
- ينفذ.
- يضع التقرير في OUTBOX.
- لا يخمن.

Morad

دوره:

Adversarial Reviewer

لا يبدأ قبل حسين.

ويقوم بـ:

Evidence

→ Challenge

→ Failure scenarios

→ Required correction

وكان الهدف أن تكون العلاقة:

Hussein

↓

OUTBOX

↓

Morad

↓

OUTBOX

↓

CTO

وليس أن يعمل الاثنان بالتوازي على نفس المهمة.

17. الخطأ في Orchestration

هنا حدث أكبر إخفاق إداري مني.

تم إعطاء Tasks مثل:

TASK-INV-003

TASK-INV-004

TASK-INV-005

لكن لم تكن آلية الـ Task نفسها مرتبطة بشكل موثوق بما يكفي بملفات العمل.

فظهر أكثر من مرة:

PHASE-1-PRODUCTION-CONTRACT.md

غير موجود في المكان المتوقع.

وهذا جعل حسين يتوقف.

ثم ظهر أن مراد يستطيع أن يرد قبل اكتمال حسين.

أي أن:

Workflow dependency

لم تكن enforced فعليًا.

18. النتيجة: المساعدون أصبحوا جزءًا من المشكلة

بدل أن يصبح الفريق:

CTO

↓

Hussein

↓

Morad

↓
Patch
↓
Tests

أصبح عمليًا:

CTO
↓
Task
↓
Missing file
↓
Blocked
↓
Evidence
↓
Another Task
↓
Another Review
↓
Blocked
↓
Consultation
↓
More Evidence

وهذه هي **حلقة الدوران** التي أشارت إليها أنت في النهاية.

19. الاستشارات الخارجية

تم طلب رأي عدة مساعدين مستقلين.

واتفق معظمهم على:

Root Cause

Schema Drift
+
Migration Discipline Failure
+
RPC Contract Drift
+

Distributed Business Logic
+
Insufficient Contract Testing

وهذا يتوافق مع التشخيص الذي كان قد ظهر قبل مرحلة الاستشارات.
لكن اختلفوا في الحل التنفيذي.

20. أهم اختلاف في الاستشارات

بعضهم قال:

SQL

ADD completed_by

وبعضهم قال:

Audit table

وبعضهم اقترح:

request_id in inventory_log

وبعضهم:

movement_registry

وبعضهم اقترح تغييرات Schema إضافية.

المشكلة في بعض هذه الردود أنها انتقلت من:

Evidence

إلى:

Architecture Decision

ثم إلى:

ALTER TABLE

بسرعة أكبر مما يسمح به التحقيق.

ولهذا لم يكن من الصحيح تنفيذ أي منها مباشرة.

21. الخطأ الأكبر في SQL المقترحة

أحد الردود اقترح:

SQL

```
ALTERTABLE stock_vouchers  
ADDCOLUMN completed_by ...
```

ثم:

SQL

```
ALTERTABLE stock_vouchers  
ADDCOLUMN last_request_id UUID
```

ثم:

SQL

```
ALTERTABLE inventory_log  
ADDCOLUMN request_id UUID
```

ثم Unique Index.

هذا يبدو احترافياً ظاهرياً، لكنه ليس كافياً.

لأن:

last_request_id

لا يمثل سلسلة Partial Receives.

و:

inventory_log.request_id

لا ينبغي إضافته قبل تحديد ما إذا كان inventory_log هو أصل Operation Identity أم مجرد Ledger للحركة.

إضافة هذه الأعمدة كانت ستعالج المشكلة بشكل محتمل، لكنها ليست مثبتة كأفضل تصميم للمشروع.

22. ما كان يجب أن يكون التصميم النهائي

كان يجب الوصول إلى:

Layer 1 — Business Contract

تحديد:

Voucher types
Voucher states
Custody
Source
Target
Partial receive rules
Cancel rules
Complete rules

Layer 2 — Inventory Engine

محرك مركزي:

post_stock_movement

Layer 3 — Operation Identity

كل Business Operation لها:

operation_id / idempotency_key

Layer 4 — Atomic Transaction

داخل PostgreSQL:

LOCK
→ Validate
→ Move
→ Log
→ Update quantity
→ Audit
→ Commit

Layer 5 — Contract Tests

اختبار:

Schema
RPC
Business Contract

معًا.

Layer 6 — Deployment Gate

لا Deploy إذا:

RPC references missing column

أو:

RPC contract != schema contract

23. Van Sales كان يجب أن يأتي بعد Inventory

لم يكن من الصحيح القفز إلى `van-sales.html`.

لأن Van Sales يعتمد على صحة:

MAIN

↓

Loading

↓

VAN custody

↓

Van Sale

↓

Customer

↓

Return

↓

Unloading

إذا كانت Inventory Engine غير موحدة، فإن إصلاح Van Sales UI أولاً سيضيف Business Logic جديدًا فوق نظام غير مستقر.

وهذا تحديدًا كان أحد أسباب قرار إيقاف التطوير على `van-sales.html`.

24. المستهدف المرحلي الصحيح

لو أعيد تنفيذ المهمة من الصفر، فالخطة الصحيحة تكون:

Phase 0 — Evidence Baseline

المستهدف:

Production Contract واحد موثق.

يشمل:

- Tables
- Columns
- Constraints
- Indexes
- RPCs
- Privileges
- RLS
- Current stock

Output: PRODUCTION-CONTRACT

Phase 1 — Inventory Domain Contract

تحديد:

- Movement types
- Voucher lifecycle
- Custody
- Partial Receive
- Cancel
- Complete
- Idempotency

Output: INVENTORY-DOMAIN-CONTRACT

Phase 2 — Inventory Engine

تثبيت:

post_stock_movement

كمصدر وحيد لتغيير المخزون.

.Output: production-ready RPC

Phase 3 — Voucher Engine

إعادة بناء:

send
receive
complete
cancel

حول Inventory Engine.

.Output: Voucher RPC set

Phase 4 — Idempotency + Concurrency

اختبارات:

Retry
Concurrent Receive
Over Receive
Partial Receive

.Output: deterministic transaction behavior

Phase 5 — Voucher UI

فقط بعد نجاح Core:

vouchers.html

Phase 6 — Van Custody

تثبيت:

MAIN → VAN
VAN → CUSTOMER

CUSTOMER → VAN
VAN → MAIN

ثم:

van-sales.html

Phase 7 — Edge Functions

إعادة توجيه العمليات إلى Engines:

receive-purchase
send-stock-voucher
receive-stock-voucher
save-sales-invoice
complete-loading
complete-return
unload-runsheet

بدل تعديل المخزون كل واحدة بطريقتها.

Phase 8 — Accounting

بعد Inventory:

post_journal_entry
post_ledger_entry

Phase 9 — End-to-End

اختبار:

Purchase
→ Branch
→ Loading
→ VAN
→ Sale
→ Return
→ Unload
→ Settlement

25. ما تم إنجازه فعليًا

رغم فشلنا في الوصول إلى Deployment، لم تكن النتيجة التقنية صفرًا بالكامل.
تم الوصول إلى:

مثبت فعليًا

- Production company context
 - Production branch context
 - Main branch
 - Actual stock state
 - Actual stock_vouchers schema
 - Actual inventory_log schema
 - Actual RPC definitions
 - RPC privileges
 - RLS investigation
 - Missing completed_by
 - Partial RECEIVE idempotency risk
 - Inventory custody ambiguity
 - Schema/RPC contract drift
 - Distributed business logic diagnosis
- وتم إنشاء Evidence files فعلية في المستودع.
-

26. ما لم يتم إنجازه

وهذا هو الجزء الأهم:

لم يتم إنتاج:

- Final Inventory Engine Patch
- Final Voucher RPC Patch

- .Final Idempotency Migration
- .Final vouchers.html
- .Final van-sales.html
- .Final Edge Function rewiring
- End-to-End Inventory Test Suite ناجحة.
- .Production Deployment
- .Patch بعد Production Verification

وبالتالي مرحلة Inventory لم تُغلق.

27. لماذا تعطلنا فعليًا؟

يمكن تلخيص التعطل في خمسة أسباب:

1. لم يكن لدينا Production Contract في البداية

فبدأنا من وثائق وتصميمات مختلطة مع Production reality.

2. الخلط بين Evidence وArchitecture Decision

مثل:

```
completed_by
branch_id
request_id
```

كل واحد منها تحول بسرعة من «ملاحظة» إلى «حل محتمل».

3. عدم وجود State Machine موحدة

Voucher lifecycle لم يكن مثبّتًا في وثيقة تنفيذية واحدة تربط:

```
type
status
custody
stock movement
audit
accounting
```


4. ضعف Orchestration

المساعدون لم يعملوا ك-pipeline حقيقي.

5. غياب Definition of Done

لم يكن هناك Gate واضح يقول:

Inventory انتهى فقط عندما تنجح كل العمليات الأساسية End-to-End.

28. نقطة مهمة جدًا اكتشفتها في مراجعة الوثائق القديمة

الوثائق نفسها ليست متطابقة تمامًا.

مثلاً:

REVIEW_START_HERE.md_00 يذكر **51 جدولاً**، بينما SYSTEM_ARCHITECTURE.md_06 يذكر **52 جدولاً**. كما توجد اختلافات في أرقام الإصدارات والحالة العامة للمشروع.

هذا يؤكد أن **المشكلة ليست محصورة في Voucher RPC**.

لدينا بالفعل تاريخ من:

Documentation Drift

+

Implementation Drift

+

Production Drift

وهذا يفسر لماذا كان الاعتماد على وثيقة واحدة أو مساعد واحد غير كافٍ.

29. الخلاصة التنفيذية النهائية

لو كنت أكتب تقرير التسليم إلى مجلس إدارة أو CTO جديد، فسأكتب:

RAWAEA ERP ليس مشروعًا فاشلاً تقنيًا، لكنه دخل مرحلة أصبح فيها استمرار التطوير أسرع من قدرة النظام على الحفاظ على اتساقه.

Inventory لم يتعطل بسبب خطأ completed_by.

بل لأن:

Architecture
↓
Business Rules
↓
RPC
↓
Schema
↓
Production

لم تكن لها سلسلة Contract واحدة قابلة للتحقق آليًا.
وكلمًا حاولنا إصلاح طبقة منفردة، ظهرت فجوة في طبقة أخرى.

30. الحكم النهائي على المرحلة

البند	الحالة
مشروع ERP الأساسي	متقدم
Architecture	موجودة
Documentation	واسعة لكن بها Drift
Production Evidence	أصبحت جيدة في نطاق Inventory
Inventory Domain	غير مغلق
Stock Engine	غير موحد بالكامل
Manual Vouchers	غير جاهزة للإغلاق
Partial Receive	خطر Idempotency مثبت
completed_by	Contract mismatch مثبت
inventory_log.branch_id	Documentation mismatch مثبت
Custody Model	يحتاج Contract نهائي
vouchers.html	لم يصل إلى حالة Production Final
van-sales.html	لم يصل إلى حالة Production Final
Edge Functions	لم تُعد هندستها بالكامل حول Engine موحد
End-to-End Tests	غير مكتملة
Production Patch	لم يُنفذ

البند	الحالة
Production Deployment	لم يحدث

النتيجة

الخطوة الأصلية كانت صحيحة في اتجاهها العام، لكن التنفيذ انحرف من "إنهاء Inventory" إلى "تحقيق متكرر في Inventory".

وهذا هو السبب الحقيقي وراء ضياع الوقت.

والأهم: لم تكن المشكلة نقص أدوات أو نقص مساعدين؛ كانت المشكلة غياب خط إنتاج هندسي صارم يحول الأدلة إلى قرار ثم Patch ثم Test ثم Gate.